Python Programming Notes

Introduction to Python

Python is a high-level, interpreted, general-purpose programming language known for its readability, simplicity, and extensive ecosystem. It was created by Guido van Rossum and first released in 1991.

Python is widely used in:

- Web Development
- Data Science
- Machine Learning
- Artificial Intelligence
- Automation
- Cybersecurity
- Cloud Computing
- Scripting

Why Python?

Python has become one of the most popular programming languages because:

- Easy to learn
- Simple syntax
- Large community
- Cross-platform support
- Extensive libraries
- Excellent AI ecosystem

Example:

```python id="wd7kcp" print("Hello World")
```

```
---

# Chapter 1: Variables and Data Types

Variables store data in memory.

```python id="jlwm58"
name = "Ebin"
age = 21
cgpa = 8.5
```
```

Python automatically determines the data type.

Common data types:

Integer

```python id="x0f80h" age = 21

```
Float

```python id="q14vdo"
price = 99.99
```

String

```python id="8i8zn5" name = "Python"

```
Boolean

```python id="8szz87"
is_active = True
```

Check type:

```python id="x8yd3e" print(type(name))

```

Chapter 2: Input and Output

Getting user input:

```python id="yfz70h"
name = input("Enter name: ")
```

Displaying output:

```python id="s9xjlwm" print("Hello", name)

```
Formatted strings:

```python id="l80f7w"
age = 21

print(f"My age is {age}")
```

Chapter 3: Operators

Arithmetic Operators

```python id="vxamdn" a = 10 b = 5

print(a + b) print(a - b) print(a * b) print(a / b)

```
Comparison Operators

```python id="yv8smd"
print(a > b)
print(a < b)
```
```

### Logical Operators

```python id="zj67si" True and False True or False not True

```

Chapter 4: Conditional Statements

Conditional statements allow decision making.

if Statement

```python id="1jq4z0"
age = 18

if age >= 18:
    print("Adult")
```

if-else

```python id="o11d9r" if age >= 18: print("Adult") else: print("Minor")

```
if-elif-else

```python id="4k2e1i"
marks = 85

if marks >= 90:
    print("A")
elif marks >= 80:
    print("B")
else:
    print("C")
```

Chapter 5: Loops

Loops execute code repeatedly.

for Loop

```python id="gvkscx" for i in range(5): print(i)

```
Output:

```text id="pwy5x5"
0
1
2
3
4
```

while Loop

```python id="j0grz8" count = 0

while count < 5: print(count) count += 1

```
break

```python id="8n5x9v"
for i in range(10):
```
```

```
    if i == 5:
        break
```

continue

```python id="5g7z1n" for i in range(5): if i == 2: continue

```

Chapter 6: Functions

Functions help organize reusable code.

Defining a Function

```python id="yajq5o"
def greet():
    print("Hello")
```

Calling a Function

```python id="4xvt4n" greet()

```
Function Parameters

```python id="7dkbr5"
def greet(name):
    print(name)
```

Return Values

```python id="3k8zbz" def add(a, b): return a + b

```
Benefits:

- Reusability
- Better maintenance
- Reduced code duplication

Chapter 7: Lists
```
```

```
Lists store multiple values.

```python id="6hpc6p"
numbers = [1, 2, 3, 4]
```

Access elements:

```python id="8hns7s" print(numbers[0])

```
Add items:

```python id="c6gjvj"
numbers.append(5)
```

Remove items:

```python id="njlwmz" numbers.remove(2)

```
Loop through list:

```python id="78s23w"
for num in numbers:
    print(num)
```

Chapter 8: Tuples, Sets, and Dictionaries

Tuple

```python id="s60t1m" data = (1, 2, 3)

```
Immutable collection.

Set

```python id="fjlwmn"
skills = {"Python", "SQL"}
```

No duplicate values.
```

Dictionary

```python id="khrg2j" student = { "name": "Ebin", "age": 21 }

```
Access value:

```python id="cr5xko"
print(student["name"])
```

Chapter 9: Object-Oriented Programming

Python supports OOP.

Class

```python id="9s2mws" class Student: pass

```
Constructor

```python id="u8wzml"
class Student:

    def __init__(self, name):
        self.name = name
```

Object

```python id="1uk3e6" s = Student("Ebin")

```
Methods

```python id="rwwu4e"
class Student:

    def greet(self):
        print("Hello")
```

Inheritance

```python id="f2a14v" class Person: pass
```

class Student(Person): pass

```
Benefits:

- Reusability
- Modularity
- Scalability

---

# Chapter 10: File Handling

Writing to a file:

```python id="k0omxa"
with open("test.txt", "w") as f:
    f.write("Hello")
```

Reading a file:

```python id="otxvvl" with open("test.txt") as f: print(f.read())

```
Applications:

- Logs
- Reports
- Configuration files

Chapter 11: Exception Handling

Errors should be handled gracefully.

```python id="kvdl3q"
try:
    x = 10 / 0

except Exception as e:
    print(e)
```

Finally block:

```python id="vrgc89" try: pass finally: print("Finished")

```
Benefits:

- Prevent crashes
- Better user experience

Chapter 12: Modules and Packages

Importing modules:

```python id="fbpbk8"
import math
```

Using modules:

```python id="hl6ttv" print(math.sqrt(25))

```
Creating custom modules:

```python id="1um9cz"
# calculator.py
```

```python id="4upifj" import calculator

```

Chapter 13: Virtual Environments

Create environment:

```bash id="zjzjlwm"
python -m venv venv
```

Activate:

```bash id="wyqff8" venv\Scripts\activate

```
Install packages:
```

```bash id="jlwm2j"
pip install requests
```

Benefits:

- Dependency isolation
- Reproducible projects

Chapter 14: Working with APIs

Sending GET request:

```python id="1owjlwm" import requests

response = requests.get( "https://api.example.com" )
```

```
JSON response:

```python id="lndhx0"
data = response.json()
```

Applications:

- Weather apps
- Chatbots
- Payment systems
- AI integrations

Chapter 15: Python for Data Science and AI

Important libraries:

NumPy

```python id="i1jjpu" import numpy as np
```

```
### Pandas
```
```

```python id="usjdyq"
import pandas as pd
```

Matplotlib

```python id="g0otk8" import matplotlib.pyplot as plt

```
Scikit-Learn

```python id="b6mtyr"
from sklearn.linear_model import LinearRegression
```

TensorFlow

```python id="nn8ixt" import tensorflow as tf

```
PyTorch

```python id="5ed7vx"
import torch
```

Applications:

- Machine Learning
- Deep Learning
- Computer Vision
- NLP
- Recommendation Systems

Chapter 16: Best Practices

Follow PEP 8 style guidelines.

Good naming:

```python id="eppf3p" student_name

```
Avoid:
```
```

```python id="h9sxd7"
sn
```

Best practices:

- Use meaningful variable names
- Write reusable functions
- Add comments when necessary
- Handle exceptions
- Use virtual environments
- Write modular code
- Follow coding standards

Summary

Python is a versatile and beginner-friendly programming language widely used in software development, automation, data science, machine learning, and artificial intelligence. Understanding variables, control structures, functions, OOP, file handling, APIs, and modern libraries provides a strong foundation for building professional applications. Python continues to be one of the most in-demand programming languages in the technology industry and serves as the backbone for many modern AI systems.